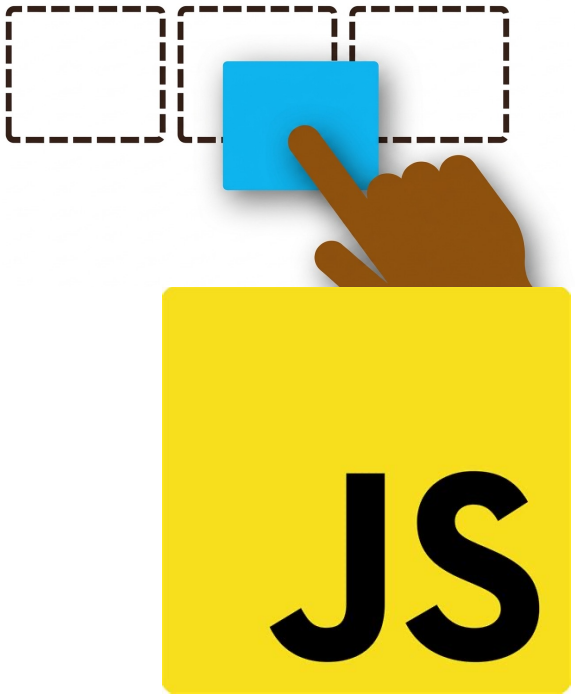


IPO

Interacción Persona Ordenador

E.T.S. Ingeniería Informática
Universidad de Sevilla

Drag&Drop API



1. Introducción
2. Diagrama de eventos
3. Gestión de eventos
4. Transferencia
5. El objeto dataTransfer
6. Atributos data-*

Introducción

La API Drag And Drop (arrastrar y soltar) permite reubicar elementos de una página web

Esta operación involucra dos tipos de elementos: origen (arrastrables) y destino de la operación.

Elementos origen (arrastrables): Son aquellos elementos HTML con un atributo draggable con valor true

```
<div draggable="true">....</div>
```

Eventos asociados:

- dragstart: Comienzo del arrastre de un elemento *draggable*
- dragend: Cancelación del arrastre de un elemento **draggable**. La gestión de este evento es opcional

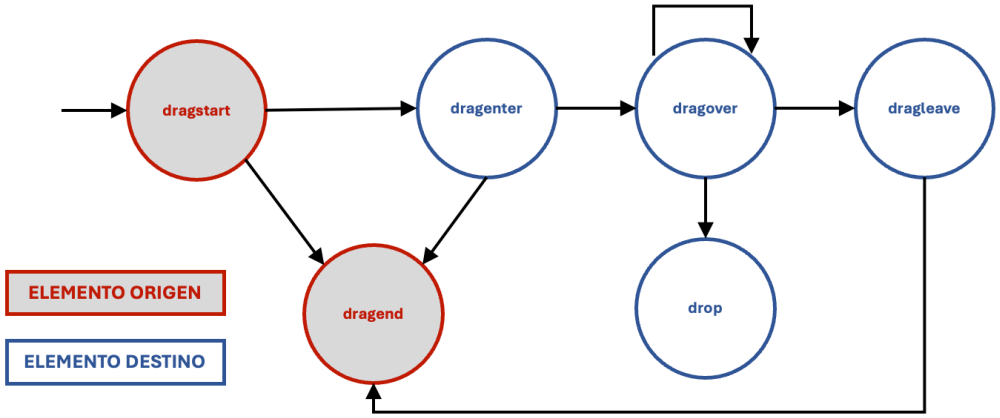
Elementos destino: Son aquellos elementos HTML que responden a los siguientes eventos:

- dragenter: Señala que un elemento arrastrable ha entrado en un elemento destino
- dragover: Indica que un elemento arrastrable esta en el interior del elemento destino
- dragleave: Señala que un elemento arrastrable ha salido dentro de un elemento destino
- drop: Indica que el elemento arrastrable se ha soltado en el elemento destino, i.e. la operación de arrastre ha concluido.

Diagrama de eventos

- Los eventos dragstar y drop determinan respectivamente el comienzo y fin de una operación Drag&Drop.
- El evento dragend se interpreta como una cancelación de la operación.
- Los eventos dragenter y dragleave indican la entrada y salida del elemento arrastrable en zona para soltar.
- El evento dragover se lanza sucesivamente mientras el elemento arrastrable está dentro de una zona para soltar.
- El evento drop, en caso de producirse, siempre sucede al evento dragover que garantiza que el elemento arrastrable está sobre un elemento destino

Diagrama de eventos *Drag&Drop*




```
// Selección de elementos interactivos
const origen = document.querySelector("[draggable='true']");
const destino= document.querySelector("[data-drop]");

origen.addEventListener("dragstart", comienzoArrastre);
origen.addEventListener("dragend", cancelacionArrastre);

destino.addEventListener("dragover", sobrevolandoDestino);
destino.addEventListener("drop", soltadoEnDestino);
destino.addEventListener("dragenter", entradaEnDestino);
destino.addEventListener("dragleave", salidaDeDestino);

function comienzoArrastre(e) {
    console.log("Comienzo de arrastre");
}

function cancelacionArrastre(e) {
  console.log("Cancelacion de arrastre");
}

function sobrevolandoDestino(e) {
  console.log("Sobrevolando destino");
  // OJO: Necesario para permitir el drop
  e.preventDefault();
}

function soltadoEnDestino(e) {
  console.log("Elemento soltado en destino");
  // OJO: Algunos navegadores responden con una redirección inmediata
  // al soltar el elemento, para prevenir este coportamiento
  e.preventDefault();
}

function entradaEnDestino(e) {
  console.log("Entrada en destino");
}

function salidaDeDestino(e) {
  console.log("Salida de destino");
}
```

Gestión de eventos

Es frecuente acompañar la operación de Drag&Dropo con cambios de estilos en los elementos involucrados para ayudar a visualizar la operación.

```
function comienzoArrastre(e) {
  console.log("Comienzo de arrastre");
  e.target.classList.toggle("elemento-en-arrastre");
}

function cancelacionArrastre(e) {
  console.log("Cancelacion de arrastre");
  e.target.classList.toggle("elemento-en-arrastre");
}

function sobrevolandoDestino(e) {
  console.log("Sobrevolando destino");
  // OJO: Necesario para permitir el drop
  e.preventDefault();
}

function soltadoEnDestino(e) {
  console.log("Elemento soltado en destino");

  e.preventDefault();
  e.target.classList.remove("elemento-en-sobrevuelo");
  e.target.classList.add("elemento-soltado");
}

function entradaEnDestino(e) {
  console.log("Entrada en destino");
  e.target.classList.add("elemento-en-sobrevuelo");
}

function salidaDeDestino(e) {
  console.log("Salida de destino");
  e.target.classList.remove("elemento-en-sobrevuelo");
}
```

- Estrategia: asociar clases CSS a los elementos origen y destino que serán manipulados en los gestores de eventos mediante la propiedad .classList.
- Salvo para cambios de estilos, no suele ser estrictamente necesaria la gestión de los eventos: dragenter, dragleave y dragend

```
.elemento-en-arrastre{
  color: var(--color-en-arrastre)
}

.elemento-en-sobrevuelo {
  color: var(--color-en-sobrevuelo);
}

.elemento-soltado {
  background-color: var(--color-en-soltado);
}
```

Hay elementos HTML que tienen una respuesta por defecto ante acciones de soltado. Para evitar estos comportamientos, la estrategia recomendada es:

- incluir preventDefault() en los gestore de eventos dragover. De esta forma informamos al elemento que debe responder al evento drop según lo indicado en el gestor definido.
- incluir preventDefault() en los gestores de eventos drop. Algunos navegadores responden de forma automática con la apertura de archivos o la redirección de la navegación. Por precaución es aconsejable añadir preventDefault() (o stopPropagation()) para evitar dicho comportamiento.

Transferencia

La operación de arrastre suele ir acompañada con transferencia de información desde el elemento origen al destino.

- un mensaje que se debe mostrar en el destino
- un cambio de estilo en función del elemento soltado

Esta transferencia involucra fundamentalmente a los gestores de los eventos siguientes

- dragstart: su gestor establece la información a transferir
- drop: su gestor recibe la información transferida para procesarla

NOTA: Para garantizar la transferencia, aunque sólo intervienen los eventos anteriores, es necesario definir el gestor para el evento dragover, mínimamente con preventDefault(), para garantizar que no se ignora el gestor para el evento drop.

La transferencia se puede realizar mediante:

- una variable global accedida en los gestores de dragstart y drop. En el primer gestor se da un valor a la variable global y en el segundo gestor se recupera el valor de la variable.
- el objeto dataTransfer (es una propiedad de event).

El método recomendado es dataTransfer

- Las variables globales no son permitidas para transferir información sobre ciertos elementos como iframe o nodos en árboles sombra
- El objeto dataTransfer es un mecanismo más seguro debido a que es un objeto temporal vinculado a la operación: se crea al comenzar el arrastre y se destruye al soltarlo.

El objeto dataTransfer

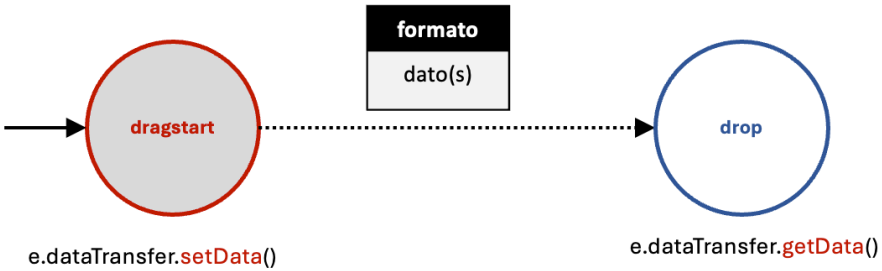
El objeto dataTransfer ofrece métodos para definir y recuperar los datos implicados en la transferencia.

- setData(formato,dato)
- getData(formato)

El objeto exige identificar el formato de los datos con notación MIME.

- text/plain para transferir cadenas (o números representados por cadenas)
- application/json para objetos

Diagrama de eventos *Drag&Drop*




```
function comienzoArrastre(e) {

  // Transferencia de cadena de texto
  const dato = "Una cadena";
  e.dataTransfer.setData("text/plain", dato);

  // Transferencia de objetos (usando JSON)
  const objeto = {
    mensaje: "Un mensaje",
    codigo: 12345,
  };

  const datoJSON = JSON.stringify(objeto);
  e.dataTransfer.setData("application/json", datoJSON);

}

function sobrevolandoDestino(e) {
  // OJO: Para poder responder al evento drop
  e.preventDefault();
}

function elementoSoltadoEnDestino(e) {
  // OJO: Para evitar la redirección inmediata de algunos navegadores
  e.preventDefault();

  // Recuperación de la información transferida
  const dato = e.dataTransfer.getData("text/plain");
  const objetoJSON = e.dataTransfer.getData("application/json");
  const objeto = JSON.parse(objetoJSON);

  const informacion=  dato + " ---  {" + objeto.mensaje + ", " + objeto.codigo+ "}";
  e.target.textContent = "Transferencia: " + informacion;
}
```

Los objetos JS deben ser transferidos mediante objetos JSON. Para realizar la conversión se usan los métodos JSON.stringfy() y JSON.parse()

Atributos data-*

Los atributos data-* son atributos personalizados que pueden figurar en cualquier elemento HTML

- Se distinguen del resto de atributos por comenzar con el prefijo data- al que le sigue un nombre
- Los atributos pueden ser booleanos (ej. data-item) o no (ej. data-id="id001")
- Como el resto de los atributos, tiene asociado el selector CSS []: (ej. [data-item] o [data-id="id1001"])
- El navegador ignora los atributos data-*, lo que significa que su uso no tiene ningún efecto visual

```html
<h1>Ver la consola</h1>
<h2 data-titulo> Lista de nombres </h2>
<ul data-lista="nombres">
  <li data-item data-id="nombre001">Pedro</li>
  <li data-item data-id="nombre002">Ana</li>
  <li data-item data-id="nombre003">Lucía</li>
</ul>
```

```js
const selectorTitulo = "[data-titulo]";
const tituloElement = document.querySelector(selectorTitulo);
console.log(tituloElement.textContent);

const selectorLista = "[data-lista='nombres']";
const listaElement = document.querySelector(selectorLista);
listaElement.classList.add("destacado");

const itemsElement = listaElement.querySelectorAll("[data-item]");

itemsElement.forEach((item) => {
  const idItem = item.dataset.id;
  console.log(idItem, item.textContent);
});
```

Son un mecanismo limpio y simple de comunicación entre HTML y JavaScript.

- Los valores del atributo permiten registrar información que JS puede necesitar para realizar ciertas funcionalidades
- Facilita el acceso desde JS a múltiples elementos


UNIVERSIDAD DE SEVILLA
UNIVERSIDAD Ð SEVILLA